PROBLEM 1 · SOLUTION

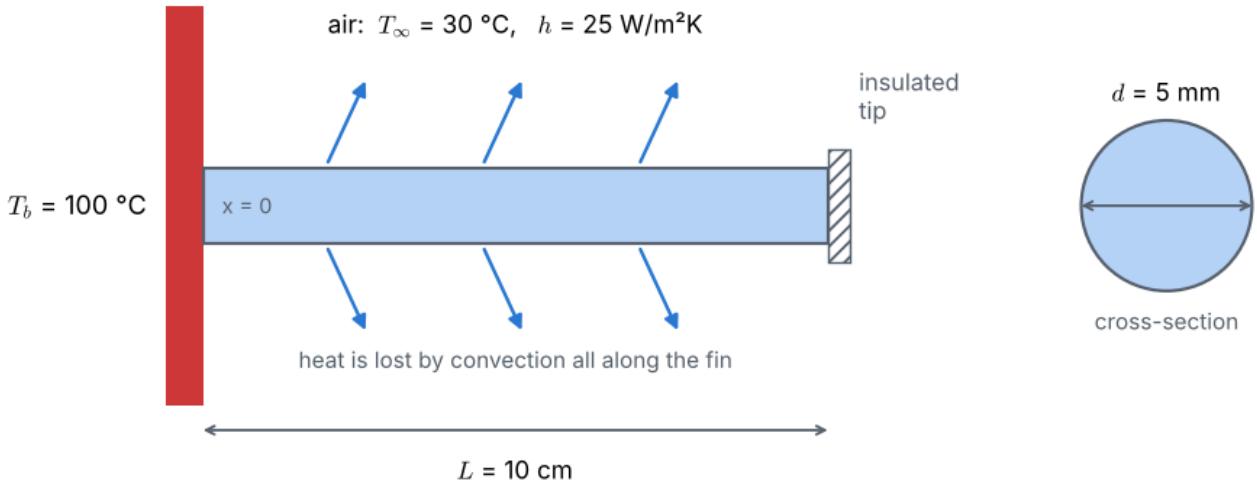
Steady temperature distribution in an aluminium pin fin

Step-by-step solution with the Finite Difference Method

Plan. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code below is plain Python (lists and loops only; `math` for the exact solution). It is the code of the notebook `03_fdm_solver.ipynb`, where it can be run cell by cell.

Step 1 · Geometry and data

A thin aluminium pin of length $L = 10$ cm and diameter $d = 5$ mm sticks out of a wall at $T_b = 100$ °C into air at $T_\infty = 30$ °C, with $h = 25$ W/(m²·K) and $k = 237$ W/(m·K). The tip is insulated.



The fin and its boundary conditions.

```
# ---- fin geometry ------------------------------------------------------
L = 0.10          # fin length                          [m]
d = 0.005         # fin diameter                        [m]

# ---- material and surroundings -----------------------------------------
k = 237.0         # thermal conductivity of aluminium   [W/(m K)]
h = 25.0          # convection heat transfer coefficient [W/(m^2 K)]
T_inf = 30.0      # temperature of the surrounding air  [°C]

# ---- boundary and initial conditions -----------------------------------
T_b = 100.0       # base temperature                    [°C]
T_init = 30.0     # initial temperature of the fin      [°C]
```

Heat is *conducted* through the cross-section area A_c and *convected* from the perimeter P:

$$A_c = \frac{\pi d^2}{4}, \qquad P = \pi d.$$

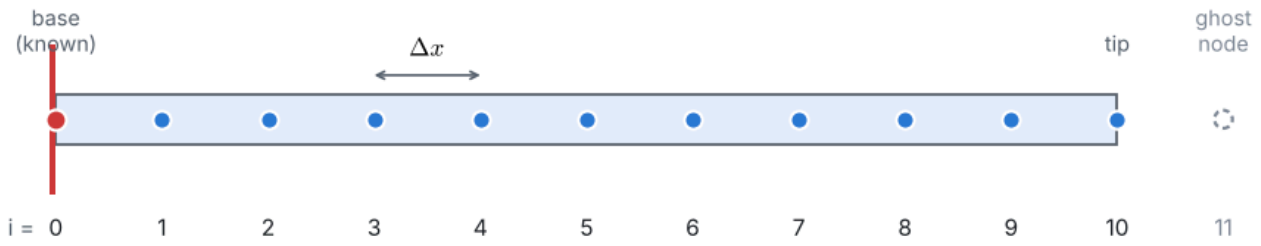
```
A_c = math.pi * d**2 / 4      # cross-section area   [m^2]
P = math.pi * d               # perimeter            [m]

print(f"cross-section area  A_c = {A_c:.4e} m^2")
print(f"perimeter           P   = {P:.4e} m")
```

```
cross-section area  A_c = 1.9635e-05 m^2
perimeter           P   = 1.5708e-02 m
```

Step 2 · The grid

We keep $N+1$ equally spaced nodes, $x_i = i\,\Delta x$ with $\Delta x = L/N$. With $N = 10$, there is one node per centimetre. Node 0 is the base (known temperature), node 10 is the tip.



The grid: 11 nodes from the base (red, known) to the tip, plus a ghost node beyond the tip.

```python
N = 10                  # number of intervals (segments)
dx = L / N              # grid spacing [m]

x = []                  # list with the position of every node
for i in range(N + 1):
    x.append(i * dx)

print(f"N = {N} intervals, dx = {dx * 100:.1f} cm, {len(x)} nodes")
print("node positions [cm]:", [round(xi * 100, 1) for xi in x])
```

Step 3 · Physics: the fin equation

Energy balance on a slice of the fin between x and $x + dx$ in steady state:

$$q_x = q_{x+dx} + h\,(P\,dx)\,(T - T_\infty).$$

With Fourier's law $q_x = -kA_c\,dT/dx$ and $q_{x+dx} = q_x + (dq_x/dx)\,dx$:

$$kA_c\,\frac{d^2T}{dx^2} - hP\,(T - T_\infty) = 0 \quad\Longleftrightarrow\quad \frac{d^2T}{dx^2} - m^2\,(T - T_\infty) = 0, \qquad m^2 = \frac{hP}{kA_c} = \frac{4h}{kd}.$$

Boundary conditions:

- base (Dirichlet): $T(0) = T_b = 100$ °C;
- insulated tip (Neumann): $\left.\dfrac{dT}{dx}\right|_{x=L} = 0$.

Exact solution (used only to check the numerical result):

$$\frac{T(x) - T_\infty}{T_b - T_\infty} = \frac{\cosh\big(m(L - x)\big)}{\cosh(mL)}.$$

```python
m = math.sqrt(h * P / (k * A_c))          # fin parameter [1/m]


def T_exact(xx):
    """Exact temperature [°C] at position xx [m] (used only to check the FDM)."""
    return T_inf + (T_b - T_inf) * math.cosh(m * (L - xx)) / math.cosh(m * L)


print(f"m = {m:.4f} 1/m,   mL = {m * L:.4f}")
print(f"exact tip temperature T(L) = {T_exact(L):.3f} °C")
```

```
m = 9.1863 1/m,   mL = 0.9186
exact tip temperature T(L) = 78.194 °C
```

Step 4 · Numerical scheme: one algebraic equation per node

Interior nodes $i = 1, \dots, N-1$. The central difference

$$\left.\frac{d^2T}{dx^2}\right|_i \approx \frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2}$$

(truncation error $\mathcal{O}(\Delta x^2)$) turns the fin equation, after multiplying by Δx^2, into

$$T_{i-1} - (2+\beta)\,T_i + T_{i+1} = -\beta T_\infty, \qquad \beta = m^2\,\Delta x^2.$$

Base node $i = 0$: $T_0 = T_b$.

Tip node $i = N$. The interior equation would need T_{N+1}, outside the fin. A central difference of the insulated-tip condition gives

$$\left.\frac{dT}{dx}\right|_N \approx \frac{T_{N+1} - T_{N-1}}{2\,\Delta x} = 0 \quad\Longrightarrow\quad T_{N+1} = T_{N-1},$$

so the **ghost node** mirrors node $N-1$, and the tip equation becomes

$$2\,T_{N-1} - (2+\beta)\,T_N = -\beta T_\infty.$$

All $N+1$ equations have the form $a_i T_{i-1} + b_i T_i + c_i T_{i+1} = r_i$: a **tridiagonal** system. We store the four coefficient lists:

```python
def build_system(N):
    """Coefficients a, b, c, r of the FDM equations for a grid with N intervals."""
    dx = L / N
    beta = (m * dx) ** 2

    a = [0.0] * (N + 1)   # coefficient of T[i-1]
    b = [0.0] * (N + 1)   # coefficient of T[i]
    c = [0.0] * (N + 1)   # coefficient of T[i+1]
    r = [0.0] * (N + 1)   # right-hand side

    # node 0, the base: T[0] = T_b
    b[0] = 1.0
    r[0] = T_b

    # interior nodes: T[i-1] - (2 + beta) T[i] + T[i+1] = -beta T_inf
    for i in range(1, N):
        a[i] = 1.0
        b[i] = -(2.0 + beta)
        c[i] = 1.0
        r[i] = -beta * T_inf

    # node N, the insulated tip (ghost node T[N+1] = T[N-1]):
    # 2 T[N-1] - (2 + beta) T[N] = -beta T_inf
    a[N] = 2.0
    b[N] = -(2.0 + beta)
    r[N] = -beta * T_inf

    return a, b, c, r
```

For $N = 10$, the equations are:

```
a, b, c, r = build_system(N)
print(f"beta = (m dx)^2 = {(m * dx) ** 2:.5f}\n")

for i in range(N + 1):
    terms = []
    if a[i] != 0:
        terms.append(f"{a[i]:+.0f}·T{i - 1}")
    terms.append(f"{b[i]:+.5f}·T{i}")
    if c[i] != 0:
        terms.append(f"{c[i]:+.0f}·T{i + 1}")
    print(f"node {i:2d}:  {'  '.join(terms):40s} = {r[i]:9.4f}")
```

```
beta = (m dx)^2 = 0.00844

node  0:  +1.00000·T0                                =  100.0000
node  1:  +1·T0  -2.00844·T1  +1·T2                  =   -0.2532
node  2:  +1·T1  -2.00844·T2  +1·T3                  =   -0.2532
node  3:  +1·T2  -2.00844·T3  +1·T4                  =   -0.2532
node  4:  +1·T3  -2.00844·T4  +1·T5                  =   -0.2532
node  5:  +1·T4  -2.00844·T5  +1·T6                  =   -0.2532
node  6:  +1·T5  -2.00844·T6  +1·T7                  =   -0.2532
node  7:  +1·T6  -2.00844·T7  +1·T8                  =   -0.2532
node  8:  +1·T7  -2.00844·T8  +1·T9                  =   -0.2532
node  9:  +1·T8  -2.00844·T9  +1·T10                 =   -0.2532
node 10:  +2·T9  -2.00844·T10                        =   -0.2532
```

Step 5 · Solution setup: two algorithms

A · Gauss–Seidel (iterative). Solve each equation for its own node, $T_i = (r_i - a_i T_{i-1} - c_i T_{i+1})/b_i$, and sweep through the nodes repeatedly, always using the newest values. The iteration starts from the **initial condition**, the fin at 30 °C, and stops when the largest change in a sweep is below a tolerance.

```
def gauss_seidel(a, b, c, r, T_start, tol=1e-6, max_sweeps=10_000):
    """Solve a_i T[i-1] + b_i T[i] + c_i T[i+1] = r_i by Gauss-Seidel iteration."""
    T = T_start[:]                # work on a copy of the initial guess
    n = len(T)
    history = [T[:]]              # temperatures after every sweep (for plotting)
    changes = []                  # largest change in every sweep

    for sweep in range(max_sweeps):
        largest_change = 0.0
        for i in range(n):
            left = a[i] * T[i - 1] if i > 0 else 0.0
            right = c[i] * T[i + 1] if i < n - 1 else 0.0
            new_value = (r[i] - left - right) / b[i]
            largest_change = max(largest_change, abs(new_value - T[i]))
            T[i] = new_value      # use the new value immediately
        history.append(T[:])
        changes.append(largest_change)
        if largest_change < tol:  # converged
            break

    return T, history, changes
```

B · Thomas algorithm (direct). Gaussian elimination specialised to tridiagonal systems: a forward sweep removes T_{i-1} from every equation, then back substitution gives the temperatures from the tip back to the base.

```python
def thomas(a, b, c, r):
    """Solve a tridiagonal system directly (Thomas algorithm)."""
    n = len(b)
    c_star = [0.0] * n
    r_star = [0.0] * n

    # forward sweep: remove T[i-1] from every equation
    c_star[0] = c[0] / b[0]
    r_star[0] = r[0] / b[0]
    for i in range(1, n):
        denominator = b[i] - a[i] * c_star[i - 1]
        c_star[i] = c[i] / denominator
        r_star[i] = (r[i] - a[i] * r_star[i - 1]) / denominator

    # back substitution: from the tip back to the base
    T = [0.0] * n
    T[n - 1] = r_star[n - 1]
    for i in range(n - 2, -1, -1):
        T[i] = r_star[i] - c_star[i] * T[i + 1]

    return T
```

	Gauss–Seidel	Thomas algorithm
type	iterative	direct
needs	initial guess, tolerance	a tridiagonal system
pros	very simple; also works in 2D and 3D	exact (up to round-off) in one pass; $\mathcal{O}(N)$ operations
cons	many sweeps on fine grids	only for tridiagonal (1D) systems

Step 6 · Compute

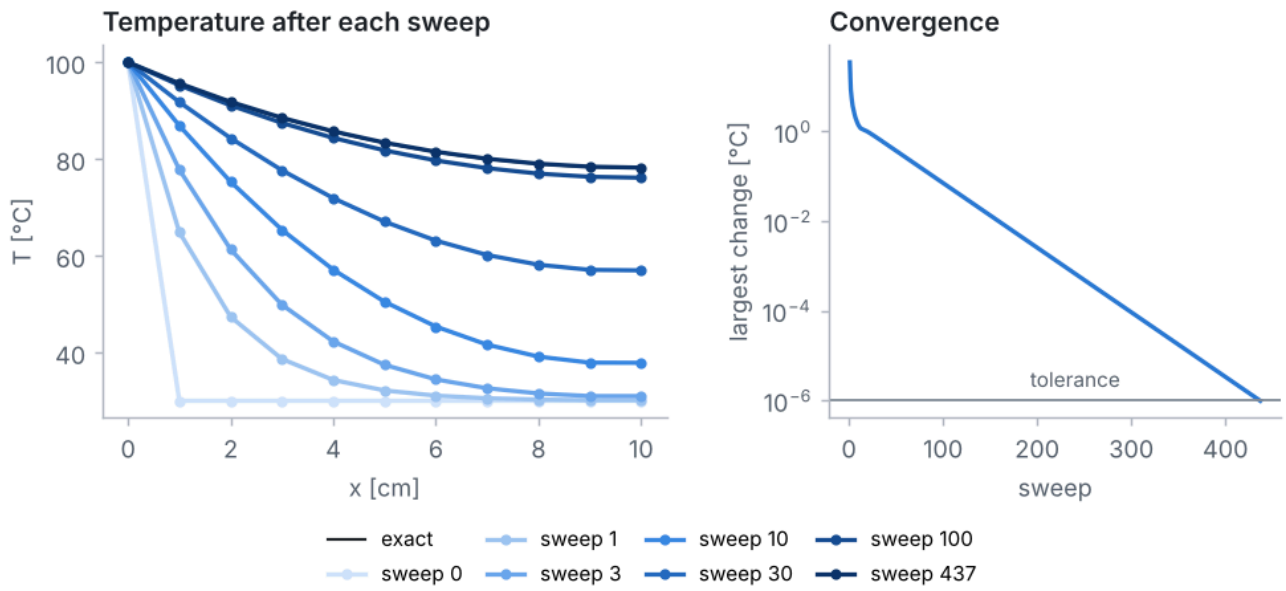
```python
tol = 1e-6                        # stop when no temperature changes by more than 1e-6 °C

T_start = [T_init] * (N + 1)      # initial condition: the whole fin at 30 °C ...
T_start[0] = T_b                  # ... except the base, which is held at 100 °C

T_gs, history, changes = gauss_seidel(a, b, c, r, T_start, tol)
T_th = thomas(a, b, c, r)

print(f"Gauss-Seidel converged after {len(changes)} sweeps")
print(f"largest difference between the two methods: {max(abs(p - q) for p, q in zip(T_gs, T_th)):.1e} °C")
```

```
Gauss-Seidel converged after 437 sweeps
largest difference between the two methods: 2.9e-05 °C
```



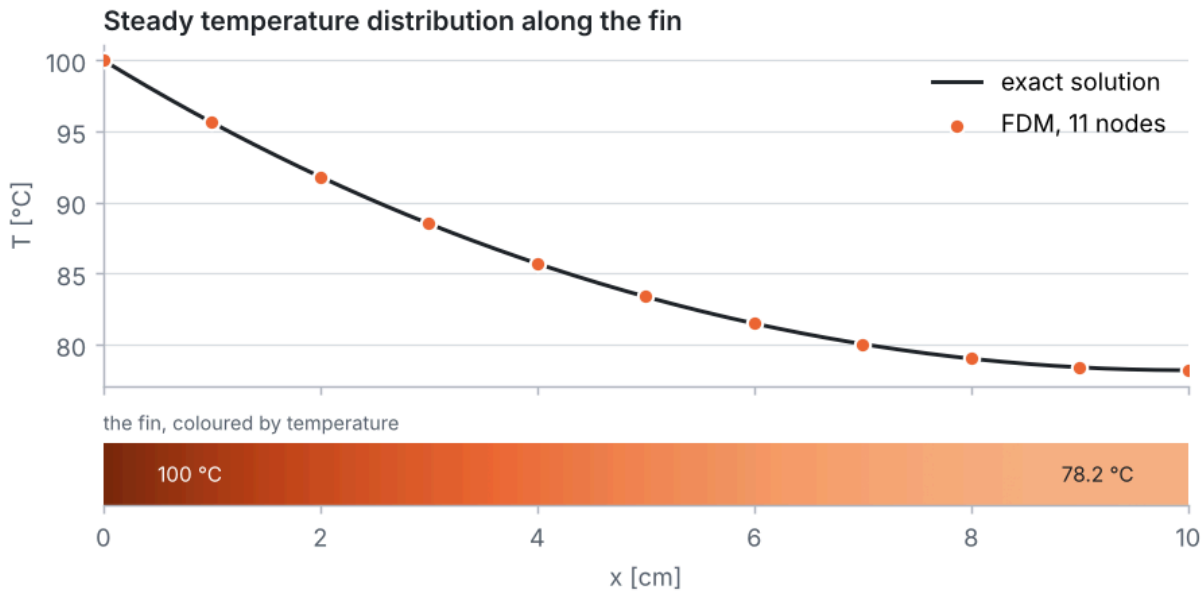
Left: temperature after successive Gauss–Seidel sweeps, starting from the initial 30 °C. Right: the largest change per sweep falls until it reaches the tolerance.

A small change per sweep is not a small error. Gauss–Seidel approaches the answer in small steps, so the tolerance must be much smaller than the accuracy you need. With a tolerance of 10^{-4} °C, the result still differs from the direct solution by about 0.003 °C; with 10^{-6} °C the two agree.

```
print(" node   x [cm]   Gauss-Seidel   Thomas    exact     error")
for i in range(N + 1):
    exact = T_exact(x[i])
    print(f"{i:4d}   {x[i] * 100:5.1f}    {T_gs[i]:9.4f}   {T_th[i]:9.4f}  {exact:8.4f}  {T_th[i] -
exact:+.4f}")
```

```
 node   x [cm]   Gauss-Seidel   Thomas    exact     error
   0     0.0     100.0000    100.0000  100.0000  +0.0000
   1     1.0      95.6278     95.6278   95.6254  +0.0024
   2     2.0      91.8094     91.8094   91.8049  +0.0045
   3     3.0      88.5126     88.5126   88.5064  +0.0062
   4     4.0      85.7095     85.7095   85.7019  +0.0076
   5     5.0      83.3766     83.3766   83.3678  +0.0088
   6     6.0      81.4941     81.4942   81.4845  +0.0097
   7     7.0      80.0462     80.0463   80.0358  +0.0104
   8     8.0      79.0206     79.0207   79.0098  +0.0109
   9     9.0      78.4087     78.4088   78.3976  +0.0112
  10    10.0      78.2053     78.2054   78.1941  +0.0113
```

Step 7 · Post-processing and verification



FDM temperatures (dots) and the exact solution (line). The strip below shows the fin coloured by temperature.

Quantities of interest. All the heat entering at the base leaves through the surface by convection. Adding the convection of every segment (trapezoidal rule) gives the heat removed by the fin:

$$q_{fin} \approx hP\,\Delta x\left[\tfrac{1}{2}(T_0 - T_\infty) + \sum_{i=1}^{N-1}(T_i - T_\infty) + \tfrac{1}{2}(T_N - T_\infty)\right], \qquad q_{exact} = \sqrt{hPkA_c}\,(T_b - T_\infty)\tanh(mL).$$

```
q_fdm = 0.5 * (T_th[0] - T_inf) + 0.5 * (T_th[N] - T_inf)
for i in range(1, N):
    q_fdm += T_th[i] - T_inf
q_fdm *= h * P * dx

q_exact = math.sqrt(h * P * k * A_c) * (T_b - T_inf) * math.tanh(m * L)

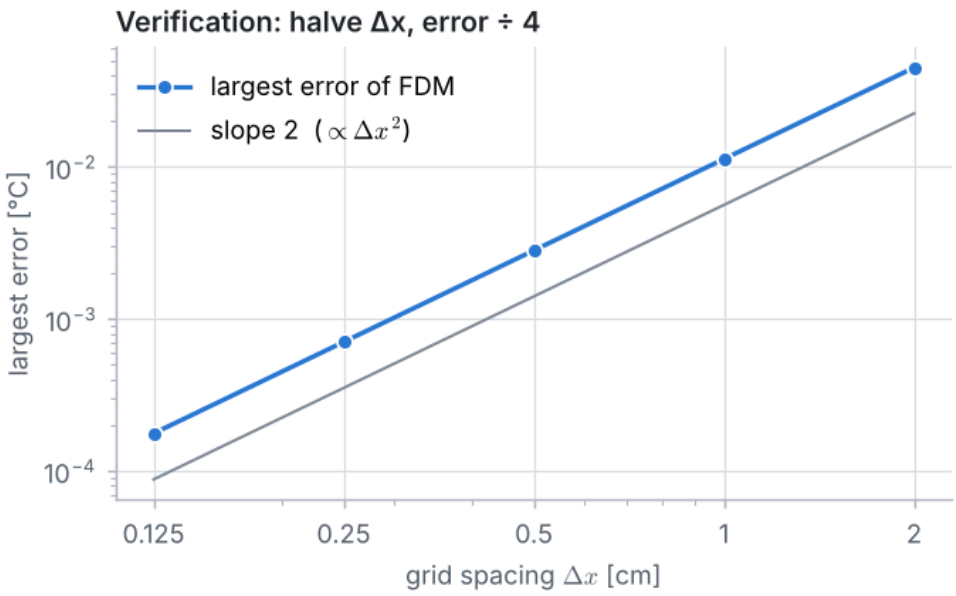
print(f"tip temperature :  FDM {T_th[N]:8.3f} °C    exact {T_exact(L):8.3f} °C")
print(f"heat loss q_fin :  FDM {q_fdm:8.4f} W     exact {q_exact:8.4f} W"
      f"    ({100 * (q_fdm - q_exact) / q_exact:+.2f} %)")
```

```
tip temperature :  FDM   78.205 °C    exact   78.194 °C
heat loss q_fin :  FDM   2.1721 W     exact   2.1702 W    (+0.08 %)
```

Grid refinement. The central difference is second-order accurate, so halving Δx should divide the error by 4:

```
dxs, errors = [], []
print("    N    dx [cm]   largest error [°C]   ratio")
for n_int in (5, 10, 20, 40, 80):
    a_n, b_n, c_n, r_n = build_system(n_int)
    T_n = thomas(a_n, b_n, c_n, r_n)
    error = max(abs(T_n[i] - T_exact(i * L / n_int)) for i in range(n_int + 1))
    ratio = f"{errors[-1] / error:5.2f}" if errors else ""
    dxs.append(L / n_int)
    errors.append(error)
    print(f"{n_int:5d}   {100 * L / n_int:7.3f}   {error:14.6f}      {ratio}")
```

```
    N    dx [cm]   largest error [°C]   ratio
    5     2.000         0.044991
   10     1.000         0.011279       3.99
   20     0.500         0.002822       4.00
   40     0.250         0.000706       4.00
   80     0.125         0.000176       4.00
```



The largest error falls along a line of slope 2: the scheme is second-order accurate.

Result. The temperature falls from 100 °C at the base to about **78.2 °C at the insulated tip**. The fin removes about **2.17 W**. With 10 intervals, the FDM solution agrees with the exact solution to about 0.01 °C, and the error falls like Δx^2.

Key ideas

- An energy balance on a small slice gives the governing equation; boundary conditions complete it.
- The central difference turns the differential equation into a tridiagonal system of algebraic equations.
- A derivative (Neumann) boundary condition is applied with a ghost node.
- Iterative (Gauss–Seidel) and direct (Thomas) solvers give the same answer; the direct one is faster for 1D problems.
- A grid-refinement study against an exact solution verifies the code and its order of accuracy.